# Relazione progetto Parallel Computing: Password Decryption

Implementazione in Python 3.14 (no-GIL) con Multithreading

Piergiorgio Danti 7196648
*Università degli Studi di Firenze*

14 aprile 2026

## Indice

**1 Abstract** **2**

**2 Introduzione** **2**
2.1 Algoritmo di brute force . . . . . 2
2.2 Sfide . . . . . 2

**3 Descrizione dell'algoritmo** **2**
3.1 Spiegazione dell'algoritmo sequenziale . . . . . 2
3.2 Identificazione delle sezioni parallelizzabili . . . . . 3

**4 Implementazione parallela** **3**

**5 Setup sperimentale** **4**

**6 Risultati Sperimentali e Analisi delle Performance** **5**
6.1 Speedup e Wall-clock . . . . . 5
6.2 Analisi di Scalabilità (Weak Scaling) . . . . . 5
6.3 Effetto del Chunk Size . . . . . 7

**7 Conclusioni** **7**

# 1 Abstract

In questo elaborato viene presentato uno studio sulla parallelizzazione di un algoritmo di decrittazione di password basato su brute force, realizzato utilizzando Python 3.14 nella versione free-threaded (senza GIL). In particolare sono state testate due diverse implementazioni per la gestione dei thread, una con la classe `Thread` del modulo `threading` e in seguito una con la classe `ThreadPoolExecutor`. A partire da un'implementazione sequenziale, utilizzata come baseline di riferimento, sono state sviluppate le versioni parallele dell'algoritmo. Le prestazioni sono state valutate sperimentalmente variando sia il numero di thread sia la dimensione dei chunk assegnati ai worker. I risultati mostrano che, nella configurazione migliore, è stato possibile ottenere uno speedup di circa 3.5 rispetto alla versione sequenziale.

# 2 Introduzione

## 2.1 Algoritmo di brute force

Per ragioni di sicurezza, le password non vengono memorizzate in chiaro ma vengono protette tramite specifici algoritmi crittografici. In generale, una password viene trasformata tramite una funzione di hashing che produce una stringa apparentemente casuale. In questo progetto è stata utilizzata la funzione di hashing fornita dal modulo `passlib.hash`. Un possibile metodo per individuare una password a partire dal suo hash è l'approccio brute force. In questo caso, un attaccante prova sistematicamente tutte le possibili combinazioni di stringhe appartenenti a un determinato spazio di ricerca, calcolando per ciascuna di esse l'hash e verificando se il risultato coincide con quello della password originale.

Nel processo di hashing viene spesso utilizzato anche un valore aggiuntivo chiamato salt, generato in maniera casuale. Il salt viene combinato con la password in chiaro e utilizzato come input per la funzione di hashing, aumentando la sicurezza del sistema. Per verificare un tentativo di password è quindi necessario conoscere anche il salt, in modo da poter ricalcolare l'hash e confrontarlo con quello memorizzato. Nel contesto di questo lavoro, per semplicità, il salt è stato considerato noto e lasciato in chiaro.

L'algoritmo di brute force consiste quindi nel generare tutte le possibili password candidate, calcolarne l'hash e confrontarlo con quello della lista di password da individuare. L'operazione di hashing ha un costo computazionale non trascurabile, poiché richiede numerosi cicli di clock. Per rendere l'esperimento gestibile in tempi ragionevoli, lo spazio delle password è stato limitato a stringhe di 8 caratteri che rappresentano date nel formato `DDMMAAAA`, comprese tra il 1950 e il 2026. Per semplicità non viene verificata la validità delle date del calendario; ogni mese viene considerato composto da 31 giorni.

## 2.2 Sfide

Dato l'elevato numero di possibili combinazioni di password e il costo computazionale dell'operazione di hashing, una ricerca puramente sequenziale può risultare poco efficiente. Tuttavia, la verifica di ciascuna password candidata è indipendente dalle altre. Per questa ragione il problema si presta naturalmente alla parallelizzazione, poiché il lavoro può essere suddiviso tra più worker che analizzano porzioni diverse dello spazio di ricerca.

# 3 Descrizione dell'algoritmo

## 3.1 Spiegazione dell'algoritmo sequenziale

Come primo passo è stata implementata una versione sequenziale dell'algoritmo. Questa implementazione esegue una ricerca brute force su un insieme di password candidate. Lo pseudocodice in 1

mostra l'algoritmo utilizzato, che continua l'esplorazione dello spazio delle password fino a quando tutte le password target sono state individuate. Per la generazione delle password, data la potenziale

**Algorithm 1** Decrittazione Sequenziale delle Password

**Require:** $hash_target$ (dizionario contenente gli hash da trovare e i relativi salt)

```
passwords ← generate_date_passwords()
passwords_trovate ← ∅
for each password ∈ passwords do
    for each (h, s) ∈ hash_target do
        if des_crypt.hash(password, s) = h then
            passwords_trovate[h] ← password
        end if
        if |passwords_trovate| = |hash_target| then
            return passwords_trovate
        end if
    end for
end for
return passwords_trovate
```

dimensione dello spazio di ricerca, la funzione `generate_date_passwords` produce le combinazioni nel range specificato senza materializzarle tutte in memoria. In particolare, utilizza il costrutto `yield` per generare le password in modo incrementale. In questo modo le password vengono prodotte solo quando necessario, evitando di saturare la memoria. Quando viene trovata una corrispondenza tra una password candidata e uno degli hash target, il risultato viene salvato in una struttura dati che mantiene l'associazione tra hash e password trovata. La ricerca termina anticipatamente non appena tutte le password richieste sono state individuate.

### 3.2 Identificazione delle sezioni parallelizzabili

La ricerca della password è un problema imbarazzantemente parallelo. Questo rende possibile suddividere lo spazio di ricerca tra più thread, ciascuno dei quali analizza una porzione distinta delle combinazioni possibili. Nel progettare la versione parallela è tuttavia necessario gestire correttamente la raccolta dei risultati parziali, evitando condizioni di concorrenza durante l'aggiornamento delle password trovate.

## 4 Implementazione parallela

Come anticipato in precedenza, sono state implementate due versioni parallele che differiscono nella gestione dei thread. In particolare, sono state utilizzate le classi:

1. `Thread`: richiede una gestione manuale del ciclo di vita del thread.
2. `ThreadPoolExecutor`: fornisce un'API di alto livello per gestire creazione, scheduling e riutilizzo di un pool di thread.

L'implementazione basata sulla classe `Thread`, che crea i worker a ogni iterazione, è stata realizzata come prima versione di riferimento per confrontare l'overhead di gestione con l'approccio più corretto ed efficiente basato sui pool di thread. Entrambe le versioni suddividono lo spazio delle possibili password in blocchi di dimensioni fissa, detti `chunk`, in modo che ciascun worker possa elaborare una porzione indipendente dello spazio di ricerca.

La funzione `check_chunk` riceve in input un insieme di password candidate, il dizionario contenente gli hash target con i rispettivi salt e un oggetto `Event`. Per ogni password del chunk viene calcolato

l'hash e confrontato con quelli del target. Durante l'iterazione viene controllato lo stato dell'oggetto `Event` e se questo risulta impostato, significa che tutte le password sono già state individuate e il worker può interrompere anticipatamente la propria esecuzione. La funzione restituisce un dizionario contenente gli eventuali elementi trovati all'interno del chunk.

La prima implementazione, `decrypt_password_par_threads`, adotta un approccio più esplicito basato sulla creazione manuale dei thread tramite il modulo `threading`. In questo caso, per ogni iterazione del ciclo principale vengono generati fino a `num_workers` chunk e per ciascuno viene creato un thread che esegue una funzione `worker`. Questa funzione invoca `check_chunk` sul chunk assegnato e aggiorna il dizionario condiviso delle password trovate.

Poiché più thread possono tentare di aggiornare simultaneamente la struttura dati condivisa, l'accesso al dizionario viene protetto tramite un oggetto `Lock`. All'interno della sezione critica il thread inserisce gli eventuali risultati trovati e verifica se il numero di password recuperate ha raggiunto il numero di hash target; in tal caso viene impostato lo `stop_event` per interrompere l'elaborazione globale. Dopo aver avviato i thread relativi ai chunk correnti, il thread principale attende la loro terminazione tramite il metodo `join`. Terminata questa fase, se la ricerca non è ancora conclusa e lo spazio delle password non è esaurito, vengono generati nuovi chunk e il processo viene ripetuto.

La seconda implementazione, `decrypt_password_par_pool`, utilizza la classe `ThreadPoolExecutor`. Vengono estratti dal generatore un numero di chunk pari al numero di worker disponibili; ciascun chunk viene quindi assegnato al pool tramite il metodo `submit`, che restituisce immediatamente un oggetto `Future` rappresentante il risultato asincrono del task.

I `Future` attivi vengono mantenuti in un insieme e la funzione `wait` viene utilizzata con la modalità `FIRST_COMPLETED` per sospendere l'esecuzione finché almeno uno dei task non termina. Quando un task termina, il risultato viene recuperato e analizzato: se sono state trovate nuove password, queste vengono aggiunte al dizionario principale, gestito dal thread master. Nel caso in cui tutte le password target siano state individuate, viene impostato l'oggetto `stop_event`, che segnala ai worker di interrompere l'elaborazione, e la funzione termina restituendo i risultati. Se invece la ricerca non è ancora conclusa, il ciclo prosegue generando nuovi chunk dal generatore e assegnandoli ai worker del pool, mantenendo così costante il numero di task attivi.

In entrambe le implementazioni lo spazio delle password non viene mai caricato completamente in memoria. Come descritto nella sezione 3, le password vengono generate in modo lazy e la funzione `itertools.islice` viene utilizzata per avere in memoria soltanto gli elementi necessari a costruire il chunk corrente.

# 5 Setup sperimentale

Le simulazioni per la raccolta delle metriche di performance sono state eseguite su un sistema dotato di processore Intel(R) Core(TM) i7-1165G7 (11th Gen). La CPU dispone di 4 core fisici e 8 thread logici grazie alla tecnologia Hyper-Threading. Il sistema presenta una gerarchia di cache composta da 320 KiB di cache L1, 5 MiB di cache L2 e 12 MiB di cache L3 condivisa. La memoria principale è di 16 GiB DDR4 a 3200 MHz. Il sistema operativo utilizzato è Linux x86_64 con kernel della serie 6.17.0.

Per valutare le prestazioni dell'algoritmo, la ricerca è stata eseguita variando sia il numero di thread sia la dimensione dei chunk. Quest'ultima rappresenta un parametro particolarmente rilevante, poiché una granularità troppo fine comporterebbe un numero eccessivo di task e quindi un alto overhead di scheduling, mentre una granularità troppo grossa produrrebbe pochi task, con il rischio di sbilanciare il carico tra i thread e ridurre l'efficienza del parallelismo.

La dimensione del chunk è stata calcolata mediante la seguente formula:

$$size = \left\lfloor \frac{N}{w \times k} \right\rfloor$$

dove $N$ rappresenta il numero totale di combinazioni da verificare (ovvero il carico di lavoro globale), $w$ il numero di thread impiegati e $k$ un fattore moltiplicativo utilizzato per controllare il numero complessivo di task generati.

Le esecuzioni parallele sono state effettuate utilizzando fino a un massimo di $8$ thread. Sono stati analizzati due scenari principali:

1. **Strong Scaling**. Il carico di lavoro è stato mantenuto costante, considerando l'intero spazio delle combinazioni di date comprese tra il 1950 e il 2026, per un totale di circa 28 mila password possibili. La ricerca è stata eseguita su tre password target distinte, una delle quali corrispondente all'ultima combinazione possibile nello spazio di ricerca, in modo da simulare il caso peggiore. Il test è stato condotto variando il numero di thread (2, 4, 6, 8) e valutando, per ciascun caso, diverse granularità dei chunk ottenute variando il parametro $k \in \{1, 10, 100, 1000\}$.

2. **Weak Scaling**. In questo scenario il carico di lavoro è stato fatto crescere proporzionalmente al numero di thread impiegati (1, 2, 4, 6, 8 thread). In particolare, l'intervallo temporale di ricerca espresso in anni è stato aumentato linearmente all'aumentare del numero di thread. Il parametro $k$ è stato fissato a $8$ ed è stata utilizzata una singola password target posizionata volutamente nell'ultima combinazione possibile ("31122026"). L'utilizzo di una sola password target consente di mantenere comparabili gli esperimenti nonostante la variazione dello spazio di ricerca.

Per verificare il corretto funzionamento dell'algoritmo, al termine di ogni esecuzione vengono stampate a video le password individuate durante la ricerca.

Come anticipato nella sezione 1, per sfruttare il parallelismo tramite thread è stata utilizzata la versione sperimentale di Python 3.14 con supporto al free-threading, che consente di disabilitare il GIL; l'ambiente è stato interamente gestito tramite uv.

# 6 Risultati Sperimentali e Analisi delle Performance

Le implementazioni descritte nelle sezioni precedenti sono state valutate attraverso una serie di test sperimentali, confrontando diverse configurazioni al fine di determinare la più efficiente.

## 6.1 Speedup e Wall-clock

Per valutare la scalabilità dell'implementazione parallela, è stato eseguito un test di strong scaling, mantenendo fisso l'intero spazio delle combinazioni di password e variando il numero di thread utilizzati. Dalla Figura 1 si osserva che, aumentando il numero di thread, le prestazioni crescono in modo quasi lineare fino a $4$ thread, avvicinandosi allo speedup ideale. Oltre questa soglia, la curva tende ad appiattirsi, segnalando il limite imposto dall'hardware. La Figura 2 mostra i tempi assoluti per le configurazioni migliori.

## 6.2 Analisi di Scalabilità (Weak Scaling)

Per l'analisi di weak scaling, il carico di lavoro (numero di anni da esplorare) è stato aumentato proporzionalmente al numero di thread, mantenendo costante il lavoro teorico per singolo worker. In questa configurazione è stata utilizzata una sola password target, poiché l'obiettivo era valutare l'efficienza

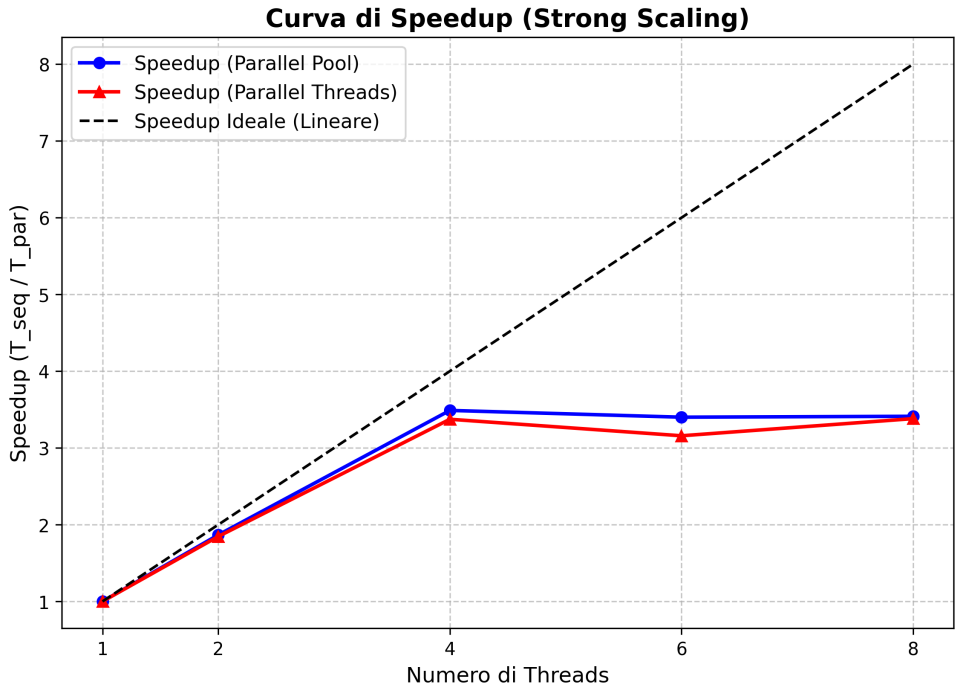


Figura 1: Curva di Speedup: confronto tra lo speedup misurato e quello ideale lineare.

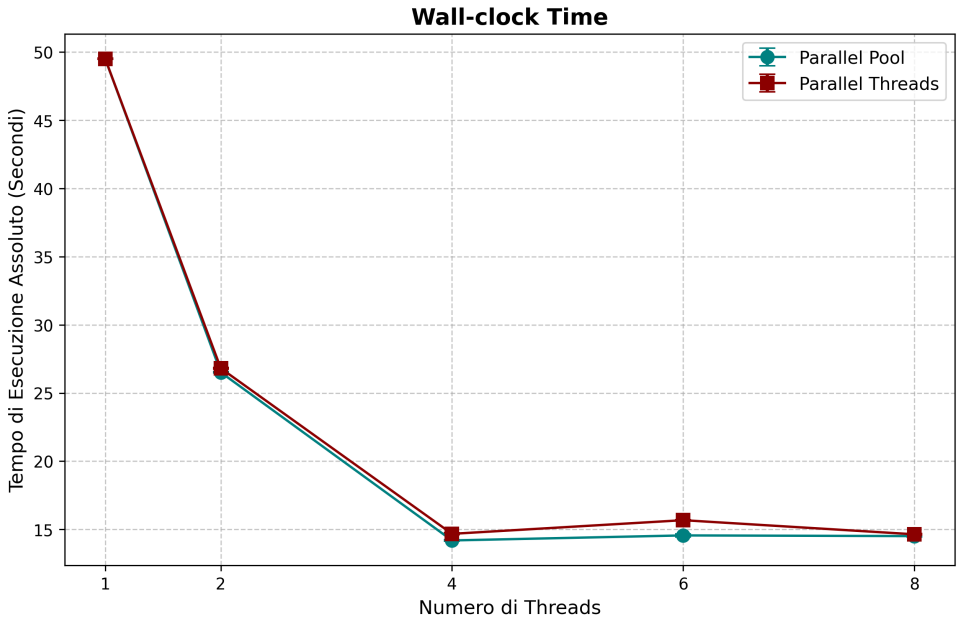


Figura 2: Wall-clock time per le migliori configurazioni.

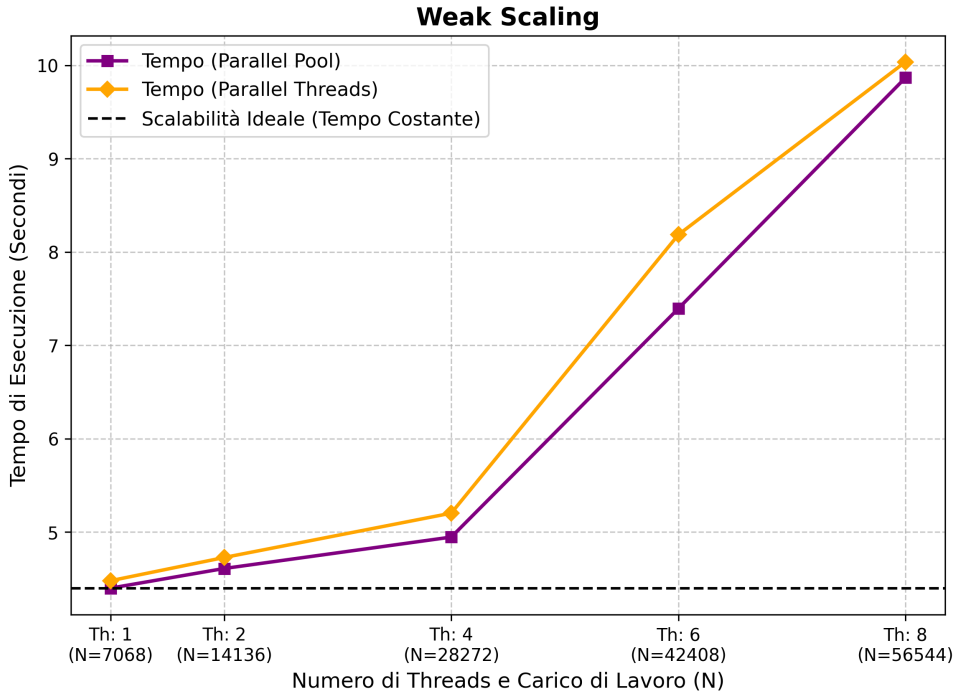


Figura 3: Analisi di Weak Scaling: tempo di esecuzione al crescere proporzionale di thread e carico di lavoro (una password).

relativa all'aumento dei thread. In uno scenario ideale senza overhead, la curva della Figura 3 sarebbe costante. Tuttavia, si nota un leggero incremento dei tempi passando da 1 a 4 thread, dovuto all'overhead di gestione. Superati i 4 thread, il tempo aumenta più significativamente, quasi raddoppiando a 8 thread, confermando che l'aggiunta di thread oltre la capacità fisica non migliora ulteriormente l'efficienza.

### 6.3 Effetto del Chunk Size

L'impatto della dimensione dei chunk è stato valutato variando il moltiplicatore $k$ nella formula 5. In generale, ci si aspetta che dimensioni troppo piccole del chunk causino overhead, mentre dimensioni troppo grandi possano rendere la ricerca e la distribuzione del carico di lavoro non ottimali.

Nell'implementazione basata su `Thread`, si osservano prestazioni migliori con dimensioni di chunk più grandi. Questo comportamento è coerente con la struttura dell'implementazione: i thread vengono creati ad ogni iterazione del ciclo principale. Tale scelta introduce un overhead aggiuntivo, il cui impatto è significativo quando la dimensione del chunk diventa estremamente piccola.

Per quanto riguarda la versione basata sulla classe `ThreadPoolExecutor`, come mostrato in Figura 4, le diverse grandezze dei chunk non hanno un impatto estremamente significativo. Comunque, come da previsione, dimensioni intermedie ci permettono di ottenere le migliori prestazioni.

## 7 Conclusioni

Tra le due versioni realizzate, quella basata sull'utilizzo di un pool di thread risulta complessivamente più efficiente, evidenziando come, in questo scenario, il riutilizzo dei thread permetta di ridurre l'overhead associato alla loro gestione. Questo consente di mantenere i worker costantemente attivi e di gestire in modo più efficiente l'esecuzione dei task. Tuttavia, mantenendo un numero di chunk pari

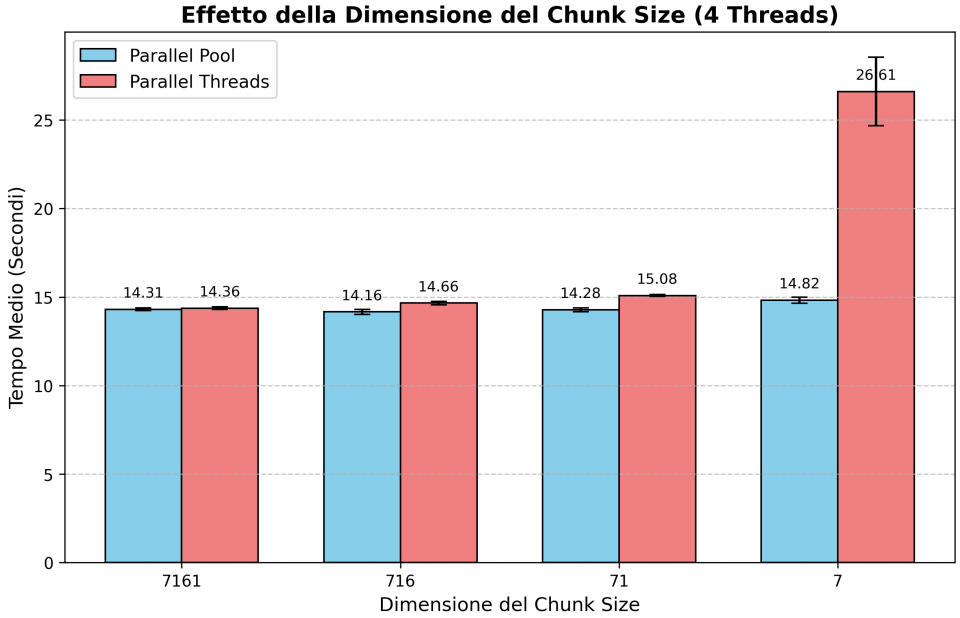


Figura 4: Effetto del Chunk Size sul tempo medio di esecuzione su circa 28.000 password.

al numero di thread l'impatto relativo di questo overhead si riduce e le prestazioni di questa implementazione possono risultare più competitive. Inoltre l'utilizzo del `lock`, per la sincronizzazione nella versione con la classe `Thread`, giustifica ulteriormente il calo di prestazioni complessivo rispetto alla versione con il pool.

Il calcolo dell'hash crittografico è un problema CPU-bound, quindi il tempo di esecuzione dipende quasi completamente dalle operazioni aritmetiche fatte dal processore. Gli accessi alla memoria sono pochi e i thread tengono le ALU praticamente sempre occupate. Per questo motivo l'Hyper-Threading non porta grandi vantaggi: i core sono già sempre impegnati a fare calcoli e non ci sono tempi di attesa da sfruttare con altri thread. Di conseguenza le prestazioni migliorano fino a circa 4 core fisici, poi la crescita si ferma. Lo speedup osservato di circa 3.5 è quindi in linea con quello che ci si aspetta da questo tipo di lavoro.

L'esperimento conferma che problemi con operazioni indipendenti tra loro beneficiano in maniera evidente del parallelismo, pur essendo limitati dall'hardware e dai costi di gestione.

## Riferimenti bibliografici


[1] Eli Collins, *Passlib: Password Hashing Library for Python*. Disponibile all'indirizzo: https://passlib.readthedocs.io/

[2] Python Software Foundation, *concurrent.futures — ThreadPoolExecutor*, Documentazione ufficiale Python. Disponibile all'indirizzo: https://docs.python.org/3/library/concurrent.futures.html

[3] Python Software Foundation, *threading — Thread-based parallelism (threading.Thread)*, Documentazione ufficiale Python. Disponibile all'indirizzo: https://docs.python.org/3/library/threading.html#threading.Thread